

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Исследование уязвимостей веб-приложений и программных интерфейсов, настройка механизмов аутентификации»

Тема *«Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»*

Выполнил:

Студент 4 курса

Группы ИИ-26

Рулько М.А.

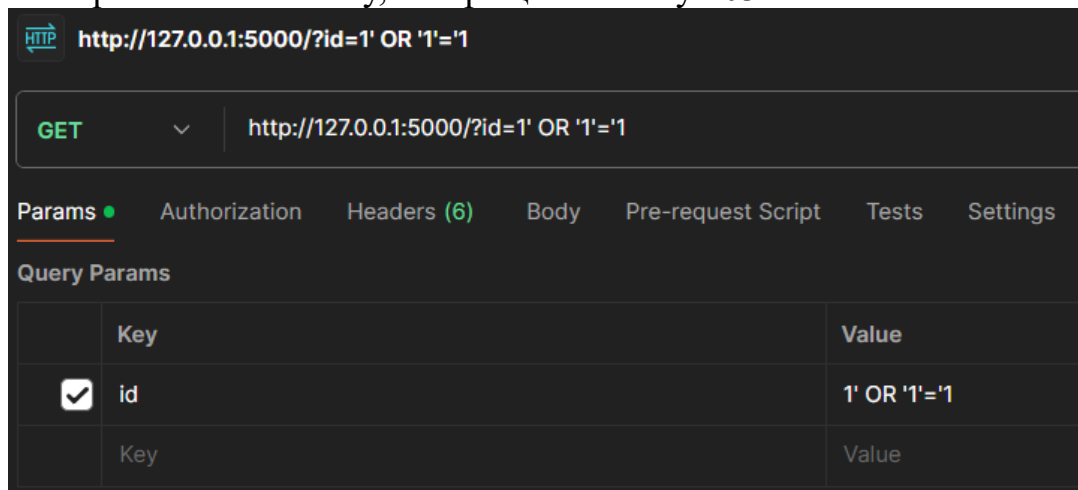
Проверил:

Хацкевич А.С.

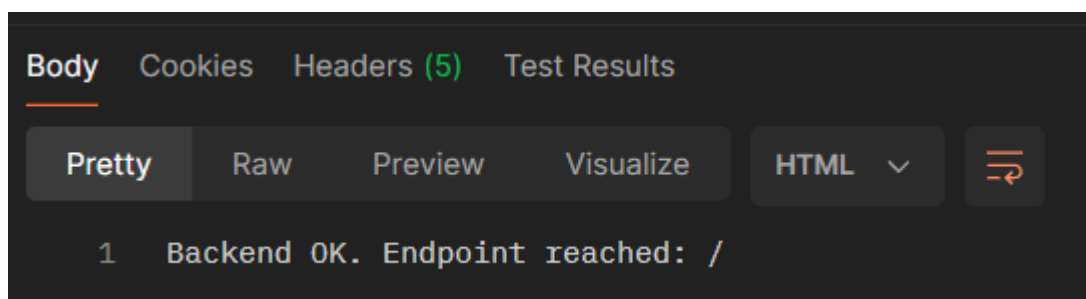
Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Задание 1. Базовая фильтрация угроз через WAF (OWASP Core Rule Set)

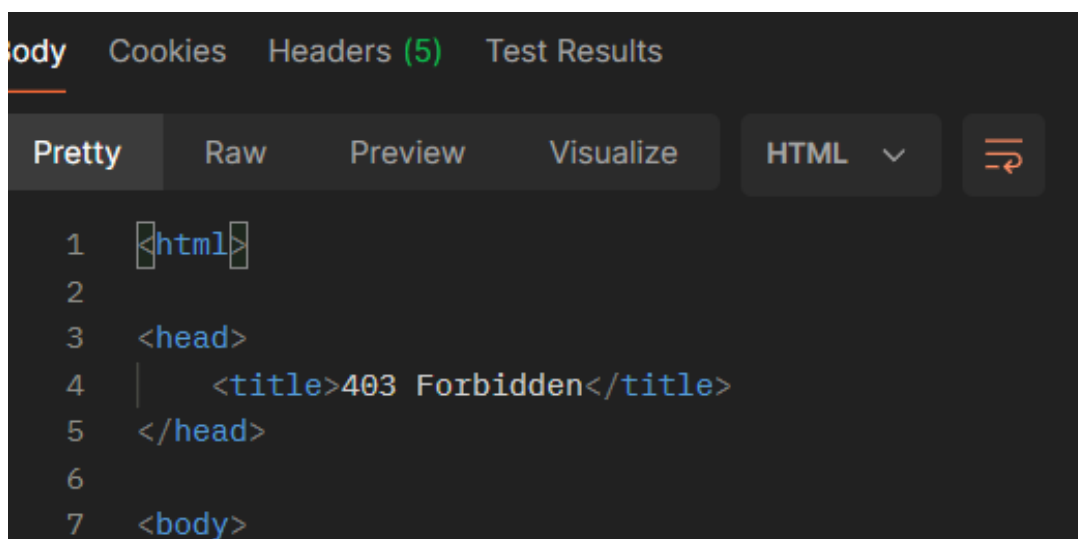
1. Выполните отправку вредоносного SQLi-запроса (например, `/?id=1' OR '1'='1`) напрямую к приложению (`http://127.0.0.1:5000`) с помощью curl и убедитесь, что приложение его обрабатывает (возвращает код 200 OK).
2. Повторите тот же SQLi-запрос через защищенный Nginx-периметр (`http://127.0.0.1`). Убедитесь, что встроенный набор правил OWASP CRS перехватывает атаку, возвращая ошибку 403 Forbidden.



Задаем тело запроса с заголовком, содержащий SQLi



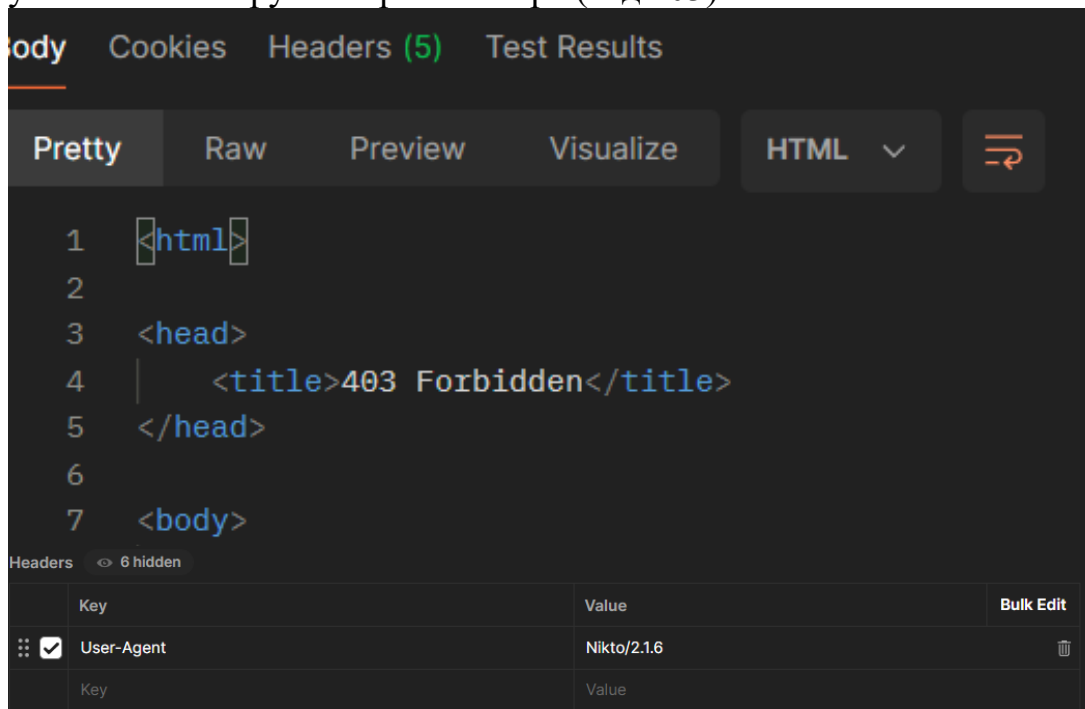
При обращении напрямую к бэкенду на порту 5000 приложение вернуло код 200 OK



При подключении по 80 порту – который слушается Nginx, получаем 403

Задание 2. Написание пользовательских правил фильтрации (L7)

1. Выполните отправку легитимного HTTP-запроса, подменив заголовок User-Agent на вредоносный сканер уязвимостей из вашего индивидуального варианта, с помощью `curl -H "User-Agent: ..."`.
2. Напишите пользовательское правило WAF в подготовленном файле `nginx/custom.conf` с присвоением уникального ID из вашего варианта. Правило должно блокировать запросы, содержащие данный User-Agent.
3. Перезапустите контейнеры (`docker-compose restart waf`) и убедитесь, что WAF успешно блокирует запрос сканера (код 403).



Наш nginx по дефолту отклоняет заголовки Nikto

```
5 SecRule REQUEST_HEADERS:User-Agent "@contains Nikto/2.1.6" \
6 "id:10002,\
7 phase:1,\
8 deny,\
9 status:403,\
10 msg:'Blocked malicious scanner Nikto'"
```

Прописываем фильтрацию явно

Задание 3. Защита от L7 DDoS-атак (Rate Limiting)

1. Выполните стресс-тест целевого эндпоинта (согласно варианту) утилитой Apache Bench, отправив 500 запросов в 10 потоков: `ab -n 500 -c 10 http://127.0.0.1<ваш_эндпоинт>`. Зафиксируйте, что сервер успешно обработал все запросы без ошибок (Failed requests: 0).
2. Сконфигурируйте директивы ограничения интенсивности трафика (`limit_req_zone` и `limit_req`) в файле `nginx/user_nginx.conf`, используя параметры `rate` и `burst` из вашего индивидуального варианта.

3. Перезапустите контейнер WAF, повторите стресс-тест утилитой ab и зафиксируйте блокировку избыточных запросов (сервер должен начать отдавать код 503 Service Unavailable для запросов, превышающих лимит).

```
C:\Users\rulko>"C:\Users\rulko\Downloads\bombardier-windows-amd64.exe" -n 500 -c 10 http://127.0.0.1/search
Bombarding http://127.0.0.1/search with 500 request(s) using 10 connection(s)
500 / 500 [=====] 100.00% 623/s 0s
Done!
Statistics      Avg      Stdev      Max
Reqs/sec      760.21    176.92    1192.77
Latency       13.24ms    2.89ms    35.94ms
HTTP codes:
 1xx - 0, 2xx - 500, 3xx - 0, 4xx - 0, 5xx - 0
others - 0
Throughput:   191.82KB/s
```

Используем утилиту *bombardier* для *DOS атаки* - успех

```
limit_req_zone $binary_remote_addr zone=search_limit:10m rate=5r/s;

server {
    listen 80;

    location /search {
        limit_req zone=search_limit burst=10 nodelay;
        limit_req_status 503;
        proxy_pass http://backend:5000;
    }

    location / {
        proxy_pass http://backend:5000;
    }
}
```

Добавили директивы для RPS – больше 5 RPS банится

```
C:\Users\rulko>"C:\Users\rulko\Downloads\bombardier-windows-amd64.exe" -n 500 -c 10 http://127.0.0.1/search
Bombarding http://127.0.0.1/search with 500 request(s) using 10 connection(s)
500 / 500 [=====] 100.00% 1247/s 0s
Done!
Statistics      Avg      Stdev      Max
Reqs/sec     1568.87    445.51    1872.83
Latency       6.31ms     3.40ms    36.98ms
HTTP codes:
 1xx - 0, 2xx - 12, 3xx - 0, 4xx - 0, 5xx - 488
others - 0
Throughput:   659.92KB/s
```

Результат работы директивы

Задание 4. Проектирование профилей защиты (Без написания кода)

1. Составьте конфигурацию директив Nginx для защиты от атаки «Slowloris» (Slow HTTP Attack), указав оптимальные значения таймаутов (client_body_timeout, client_header_timeout) и обоснуйте свой выбор.
2. Спроектируйте архитектурную схему применения CDN (Content Delivery Network) в связке с WAF для защиты от объемных (L3/L4) DDoS-атак.

```
4
5 server {
6     listen 80;
7
8     client_header_timeout 10s;
9     client_body_timeout 10s;
10    keepalive_timeout 15s;
11    send_timeout 10s;
12
13    proxy_pass http://backend:5000;
14 }
```

Директива	Значение	Обоснование
client_header_timeout	10s	Slowloris намеренно тянет передачу заголовков. Если за 10 сек заголовки не получены — соединение рвётся
client_body_timeout	10s	Защита от медленной передачи тела запроса. 10 сек достаточно для легитимных клиентов
keepalive_timeout	15s	Ограничивает время жизни keep-alive соединения, не давая атакующему держать его бесконечно
send_timeout	10s	Если клиент не читает ответ 10 сек — соединение закрывается

Описание схемы CDN + WAF

Для защиты веб-приложения от DDoS-атак применяется двухуровневая архитектура, в которой трафик последовательно проходит через CDN и WAF перед тем как достичь сервера приложения.

На первом уровне располагается CDN, который принимает весь входящий трафик. Его основная задача — отражение объёмных атак на сетевом и транспортном уровнях (L3/L4). CDN фильтрует вредоносные пакеты по IP-адресам, блокирует SYN-флуд и UDP-флуд, а также позволяет применять геоблокировку по странам. За счёт распределённой инфраструктуры CDN способен поглощать атаки большой мощности, не пропуская их дальше.

На втором уровне находится WAF, который работает на прикладном уровне (L7) и анализирует уже HTTP-трафик. WAF проверяет заголовки, параметры запросов и тело на соответствие правилам безопасности. Здесь блокируются SQL-инъекции, XSS-атаки, запросы от вредоносных сканеров и превышение допустимой частоты запросов (Rate Limiting).

До сервера приложения доходит только трафик, прошедший оба уровня проверки. Такой подход позволяет разделить ответственность между уровнями защиты и снизить нагрузку на каждый из них.

Вывод:

Разобрался с базовыми способами фрода веб приложения.